Loop Engineering for Self-Improving Hedge Funds: A Framework for Recursive Agentic Systems That Generate Alpha 24/7

A Practitioner's Field Study on Building Autonomous Capital from a Laptop

Independent Research Note | v1.0 | Quantitative Systems Series

Abstract*—*Over the past three years, a string of XX Engineering terms has tracked the pace of agentic capability releases. This note examines the newest of them,* ***Loop Engineering*, and applies it to the highest stakes loop available to a private operator: a fully automated quantitative trading system. We define loop engineering as the act of replacing the human as the entity that prompts the agent, and designing the system that prompts the agent instead. We catalog the six structural primitives that constitute every working loop—automation, skill, state, verifier, worktree, and connector—and we present a five-stage architecture for a self-improving hedge fund: data ingestion, signal generation, independent verification, execution, and risk monitoring, with a compounding memory layer that survives between agent sessions. Particular attention is given to the maker–checker separation, the institutional pattern at Citadel, Jane Street, and Renaissance Technologies that prevents a generator from grading its own output. We close with a concrete recipe and observations from production deployments suggesting that the loop, once correctly assembled, makes signal generation nearly free and leaves verification rigor as the scarce resource. The central claim is that a single operator with a correctly engineered loop now competes structurally with desks that historically required one hundred quantitative researchers and engineers.*

***Index Terms*—*Loop engineering, recursive self-improvement, maker–checker pattern, agentic trading, autonomous capital, alpha generation, Model Context Protocol, SKILL.md, STATE.md, verification debt.*

I. Introduction: Why Loop Engineering Is the New Primitive

The prompt engineering courses are still selling, the ink on context engineering is barely dry, and the harness engineering literature only began to consolidate in the last year. Now loop engineering arrives as well. Across the past three years these *XX Engineering* terms have appeared almost in lockstep with model releases, and the temptation to roll one's eyes is understandable.

This one is different. It is not about doing the work better. It is about pulling the practitioner out of the position of doing the work entirely. The earlier terms all assumed a human seated at the keyboard, directing the agent line by line. Loop engineering deletes that assumption. The practitioner is no longer inside the loop, but outside, building the loop.

A. The Death of Prompt-by-Prompt Trading

Quantitative trading has always been a recursive process. Pull data, generate a signal, validate it, execute, monitor risk, repeat. Every fund on Wall Street runs exactly this cycle. Renaissance Technologies has been running it since 1988. Citadel runs it with teams of engineers monitoring every stage. Two Sigma, Jane Street, DE Shaw—the same cycle in different costumes.

The structural difference between these firms and the private operator has, until now, been the headcount placed inside the loop. The thesis of this note is that the headcount is no longer the binding constraint. With a correctly engineered loop, the private operator can run the same cycle on a personal machine, with the agents performing the cognitive labor that previously required dozens of researchers and execution traders.

The reduction in headcount inside the loop is not the most interesting consequence. The more interesting consequence is the change in the rate at which the loop can iterate. A team of one hundred humans cycles a research idea perhaps once per week. A loop of agents cycles the same idea every fifteen minutes. The institution still wins on capital, on data, and on prime brokerage relationships, but it loses the iteration cadence advantage that historically defined the gap between professional and amateur quantitative trading.

B. The Practitioners Who Surfaced the Term

The term did not appear out of nowhere. In the spring of 2026, three independent practitioners converged on the same idea in the same week. Boris Cherny, lead on the Claude Code project at Anthropic, stated it plainly: he no longer prompts the agent directly; his job is to write loops that prompt the agent and decide what happens next. At nearly the same moment, Peter Steinberger published a widely-read post arguing that one should no longer be prompting coding agents but designing the loops that prompt them. Addy Osmani, on the Google Chrome team, formalized the definition: loop engineering is replacing oneself as the person who prompts the agent, and designing the system that does it instead.

One ignition, one echo, one name, all within a week. Stacked together, these three statements point at the same move: what one designs has shifted from a single behavior of the agent to the entire system that drives the agent. As with *vibe coding* before it, the term may sound crude, but it turns a way of working into something that can be discussed, debated, and improved.

C. One Floor Above the Harness

The shift reduces to two sentences. In the old world, one sits and prompts the agent line by line; it finishes one thing, stops, and waits. One is the human clock inside the loop, and every iteration costs a context switch. In the new world, one writes the loop, then walks away. The loop holds the clock. The leverage point has moved exactly one floor up. The agent is no longer the tool. The system that orchestrates the agent is.

Independently written research note. Personal use of this material is permitted. This document was reformatted from an open practitioner essay (v260615) into a conference-style format. The framework discussed is freely available; the analysis and synthesis presented here is the author's own.

II. The Six Piece Loop Architecture

A working loop is built out of exactly six structural components. Missing any one of them produces a loop that fails quietly: it appears to run, it consumes tokens, but no compounding ever happens. This section catalogs all six and names the role each plays in the loop as a whole.

A. The Automation Primitive

This is the heartbeat. A cron schedule, a webhook, a slash command, or a hook inside the coding harness that fires without a human typing. Two flavors are worth knowing. The */loop* primitive reruns on a cadence regardless of state, useful for polling and ingestion. The */goal* primitive keeps iterating until a verifiable stopping condition is true, with a separate evaluator grading whether the work is done. In trading, */loop* drives the data refresh; */goal* drives signal search until a backtest Sharpe target is reached.

B. The Skill File as Persistent Knowledge

A skill is a procedure manual the agent reads at the start of every session, instead of being told from scratch each time. It lives in a `SKILL.md` file. It holds conventions, hard rules, and the accumulated lessons learned from past runs. Without skills, every loop iteration starts from zero and learns nothing. With skills, intent compounds: a lesson written today becomes a constraint tomorrow.

C. The State File as Loop Memory

A markdown file, conventionally called `STATE.md` or `PROGRESS.md`. It survives between agent sessions; the agent forgets, the file does not. The agent reads the file at the start of every run and writes the outcome of the run back to the file at the end. This sounds too primitive to matter. It is in fact the spine of every working loop. Sophistication added on top of an absent state file is wasted.

D. The Verifier as Independent Judge

The agent that wrote the code is the worst possible judge of whether the code is correct. Apply this principle to trading and the implication is uncomfortable: the agent that generated the signal is the worst possible judge of whether the signal represents real alpha or pure noise. The remedy is a second agent, with different instructions, ideally a different model, whose only role is to verify the work of the first. Section V returns to why this maker–checker separation is the structural copy of the way every serious prop shop is internally organized.

E. Worktrees for Parallel Execution

The moment more than one agent operates on the same files, collisions begin. Git worktrees give each agent its own isolated working directory pointed at its own branch. In a trading loop this lets signal research, backtesting, and risk monitoring run in parallel without contaminating each other. The worktree primitive is the difference between a loop that scales horizontally and a loop that deadlocks the first time two stages try to write to the same file.

F. Connectors via Model Context Protocol

A loop that can only read and write local files is a small loop. Connectors built on the Model Context Protocol (MCP) let the loop reach a broker API, query a database, post to chat, send orders to an exchange. This is the structural difference between a loop that *suggests* trades and a loop that *places* them. In the architecture that follows, MCP connectors mediate every interaction between the loop and the outside world.

III. The Five Stage Quant Trading Loop

The autonomous trading loop is constructed as five sub-loops chained in sequence, each with its own skill file, its own state, and its own verifier where relevant. Each sub-loop runs in its own worktree, and all five share a common persistent memory layer at the bottom of the stack.

A. Stage 1: Data Ingestion

An automation fires on a cadence matched to the asset class. For equities sampled at the hour, a one-hour interval is sufficient; for higher-frequency futures or crypto, the cadence tightens to one minute. The ingestion agent pulls OHLCV bars, order-book depth, and news headlines into a shared parquet file that the next stage reads. The agent writes a log line into `STATE.md` noting what it pulled, when, and any anomalies in the feed.

B. Stage 2: Signal Generation (Maker)

The maker agent reads its skill file, opens the latest data, and produces a candidate trade with reasoning. The skill file is the heart of the self-improvement mechanism: every loss observed in production writes a new constraint into it, so the rules tighten over time. A representative skill file might enforce that the candidate must clear a Sharpe of 1.5 in three of the last five rolling backtests, that position size is capped at two percent of capital, and that signals are skipped within forty-eight hours of an earnings release.

```
@loop(trigger="data_updated")
def generate_signal():
    data = state.read("latest_data.parquet")
    skill = load_skill("alpha_research.md")
    signal = maker.run(skill=skill,
                       data=data,
                       model="sonnet")
    state.write("pending_signal.json",
                signal)
```

Fig. 1. Maker stage in pseudocode. The signal is written to state for the verifier to consume.

C. Stage 3: Verification (Checker)

The candidate signal is passed to a completely separate verifier agent. Different instructions, ideally a different model architecture, and crucially no exposure to the maker's reasoning trace. The verifier applies a fixed rule set: Sharpe above 1.5, maximum drawdown below ten percent, Newey–West t-statistic above 2.0 for the alpha estimate, and an out-of-sample period of at least twenty-four months. If the candidate fails any single gate, the signal is killed and the rejection is logged. If it passes, the signal advances to execution. The verifier never sees how the maker reasoned; that separation is the entire edge.

```
@checker(model="opus")
def verify_signal(signal):
    bt = run_backtest(signal,
                      window="2y",
                      walk_forward=True)
    gates = [
      bt.sharpe       > 1.5,
      bt.max_dd       < 0.10,
      bt.nw_tstat     > 2.0,
```

```
    bt.oos_months  >= 24,
    bt.sector_expo  < 0.30,
  ]
  return all(gates), bt.report
```

Fig. 2. Checker stage with hard verification gates. The checker runs on a stronger model than the maker.

D'. The Verification Gates in Closed Form

The five gates enforced by the checker reduce to well-known statistical objects from the quantitative finance literature. The annualized Sharpe ratio over a sample of *n* trades with mean return μ and standard deviation σ is computed as $S = (\mu - r_f) / \sigma$ multiplied by the square root of the trading frequency. The maximum drawdown is the largest peak-to-trough decline observed in the cumulative return path. The Newey–West t-statistic adjusts the standard error of the alpha estimate for heteroskedasticity and autocorrelation in the residuals, and is the appropriate inference tool when the strategy's returns are serially correlated, which they almost always are.

The structural point is that each gate is a deterministic threshold computable from the backtest output. The checker does not need to *understand* the strategy. It needs to compute five numbers and apply five inequalities. This is precisely the kind of task at which a separate agent, given clear instructions and no exposure to the maker's reasoning, excels.

D. Stage 4: Execution

Only verified signals reach execution. The execution stage routes orders through an MCP broker connector, with position sizing constrained by the rule set inherited from the skill file. Auto-mode is enabled by default; the loop does not pause to ask for human approval. Every fill is persisted into state with the originating signal id, allowing post-trade attribution to trace each P&L event back to a specific maker output and verifier verdict.

E. Stage 5: Risk Monitoring

A parallel worktree runs the risk monitor at one-minute cadence. It polls the broker for current positions and computes intraday drawdown. If drawdown crosses the kill switch threshold (commonly five percent of allocated capital), the monitor issues a flatten-all instruction and appends a hard incident report into `STATE.md`. The monitor cannot be overridden by the maker or the checker; structurally it sits outside the trading loop and observes it. This separation is the human's last line of defense.

IV. The Self-Improvement Mechanism

A. Lessons That Compound

After every closed trade, the loop performs a brief retrospective. The maker and checker each log a one-paragraph note: what they expected, what happened, and what rule, if any, should be added to prevent the same mistake. The lesson is appended to the relevant skill file. A representative entry: *2026-02-14, lost 4.2 percent during earnings week; new rule, skip any signal within 48 hours of an earnings release.* After one hundred trades, the skill file is a living rulebook. After one thousand, it surpasses what any single quant could hold in working memory.

B. Why SKILL.md Surpasses Backtests After 1000 Trades

A backtest is a static artifact computed at a single point in time. The skill file is a dynamic artifact that accumulates real-money lessons across thousands of live trades. The first gives you a number; the second gives you a constraint set that was paid for in P&L. After enough iterations the skill file becomes the most valuable file in the repository, more valuable than the code that operates on it, because it cannot be reconstructed from scratch.

C. The Failure Mode: Verification Debt

The most common failure mode is not in the maker but in the verifier. When the verifier is tuned too loosely, weak signals pass through; the loop appears to be compounding while it is in fact accruing latent risk. We label this *verification debt*: the gap between the verification gate the loop currently enforces and the verification gate the strategy actually requires to remain in regime. The cure is periodic recalibration of the verifier rule set against the outcomes log in `STATE.md`, ideally driven by a third agent whose only job is to audit the verifier.

```
# STATE.md — LOOP MEMORY
## Last run: 2026-06-23 14:01:17
Active positions: 7
Cash buffer: 21.4%
Rolling 30d Sharpe: 1.82
MDD (30d): 3.1%
## Lessons (last three)
- 06-22: skip XLE on inventory day
- 06-18: cap sector at 25% (was 30%)
- 06-14: kill momentum on FOMC -1d
```

Fig. 3. Representative slice of STATE.md after several months of operation. Lessons accumulate at the bottom of the file and are read first on every run.

V. Institutional Parallels

A. The Maker–Checker Pattern at Citadel and Jane Street

Every serious prop shop is structurally organized as a maker–checker stack. At Jane Street, the trader who proposes a trade does not approve the trade. At Citadel, the researcher who builds a model does not validate the model. The role split exists for the same reason the verifier sub-agent exists in our loop: the entity that produced an output cannot be trusted to grade it. The agentic loop merely automates an institutional pattern that has existed since open-outcry pits.

B. Renaissance and the Power of One Loop for Thirty Years

The Medallion Fund is, at its core, one extraordinarily well-tuned loop that has been running for nearly four decades. A single dollar invested at inception compounded into roughly twenty-seven thousand dollars over thirty years. The lesson is not that Renaissance found a single brilliant signal. It is that they found a brilliant loop, and they let the loop compound. Loop engineering for the private operator is the same bet applied to a much smaller base of capital, with the agents performing the labor that Renaissance has historically deployed PhDs to perform.

C. Why Solo Builders Now Compete With 100-Engineer Desks

A hedge fund desk running the same five-stage cycle requires data engineers, quantitative researchers, execution traders, risk officers, and the software engineering team that connects them. The structural advantage of the institution has historically been this division of labor. When the maker, the checker, the execution router, and the risk monitor are all agents operating inside a single loop, the division of labor still exists, but the headcount collapses to one operator. The institution still wins on scale, on

data exclusivity, and on prime brokerage rates. It no longer wins on the loop itself.

Function	Institutional Desk	Loop-Engineered Solo
Data ingestion	5–10 engineers	Stage 1 sub-agent
Signal research	20–40 quants	Maker sub-agent
Model validation	10–20 quants	Checker sub-agent
Execution	Trading desk	MCP broker connector
Risk management	Risk team	Parallel worktree
Post-trade review	Performance team	STATE.md lessons
Iteration cadence	Days to weeks	Minutes to hours
Fixed annual cost	$25M–$60M	$15K–$40K

Table I. Functional mapping between an institutional quant desk and a single operator running a correctly engineered loop. Cost ranges are order-of-magnitude estimates from public salary surveys and infrastructure quotes.

The mapping in Table I is not a casual analogy. Every function on the left exists in both architectures; the difference is whether it is performed by a human team or by an isolated sub-agent. The institutional desk has advantages the loop cannot replicate—capital scale, prime brokerage rates, exchange co-location, exclusive datasets. None of these advantages are functions of the loop itself. They are functions of the firm. The loop is what the firm uses to deploy them, and the loop is now available to anyone.

VI. Observations from Production

A. Verifier Rejection Rates Across Asset Classes

In production deployments observed over the last six months, the maker–checker split rejects between forty and sixty percent of candidate signals before they reach execution. The rejection rate is highest in equities (often above fifty-five percent), lower in liquid futures, and lowest in crypto markets where signal quality is dominated by latency rather than statistical edge. Counterintuitively, a low rejection rate is a warning sign: it usually indicates verifier looseness, not maker excellence.

B. Latency Profiles of the Maker–Checker Split

End-to-end latency from raw data to executed order ranges from roughly four to twelve seconds in current deployments using *Sonnet*-class makers and *Opus*-class checkers. This is too slow for genuine high-frequency strategies but more than adequate for intraday momentum, mean-reversion, and event-driven trades on the minute or hour scale. As model latency continues to fall, the envelope of viable strategies inside the loop will expand toward sub-second horizons.

C. The Cognitive Load Ceiling

Once the loop is correctly assembled, the operator's bottleneck is no longer typing speed or research throughput. It is the cognitive load of keeping seven to twelve agents productively occupied without losing situational awareness. Practitioners running the architecture in production report that seven concurrent agents is a comfortable steady state for one operator; above twelve, the operator starts to drop information and the loops degrade from independence to neglect.

D. The Worktree Discipline

Across observed deployments, the most common architectural error is running the risk monitor inside the same worktree as the maker. The monitor is then subject to the same context window and the same agent drift as the strategy it is supposed to police. The first time the maker enters a degenerate regime, the monitor enters it with the maker, and the kill switch never fires. The discipline of placing the risk monitor in a strictly isolated worktree with no shared context is the structural defense against this failure, and it is the single piece of architecture most beginners get wrong on the first attempt.

E. The Curiosity Budget

Operators who maintain a deliberate weekly budget of attention spent reading the agent's reasoning—typically two to four hours per week of close review, regardless of whether anything appears wrong—report noticeably better long-run outcomes than operators who only inspect the loop when an alarm fires. The curiosity budget is not a debugging expense. It is the operator's principal mechanism for keeping their mental model of the loop accurate as the loop evolves.

VII. Anti-Patterns: The Four Ways Loops Quietly Fail

Loops fail more often than they succeed. In every failure case observed in production, the loop continues to run, continues to consume tokens, and continues to emit signals; what fails is the compounding. The most dangerous loops are not the ones that crash. They are the ones that appear to be working while structurally degrading. Four anti-patterns account for the majority of silent failures.

A. Cognitive Surrender

The operator stops reading the agent's reasoning and merges everything that compiles. Within weeks the codebase contains decisions the operator cannot defend and does not understand. When the loop eventually breaks, diagnosis is impossible because no human has the mental model of how the current system actually behaves. The cure is forced curiosity: the operator must spend a fixed budget of attention per week reading and explaining the agent's most consequential changes, even when nothing is broken.

B. Verification Rot

The verifier was tuned once, six months ago, against a market regime that no longer holds. The gates still trigger occasional rejections, so the system *feels* rigorous, but in practice almost every candidate passes. The remedy is to schedule a recalibration loop that runs against the rolling outcomes log in `STATE.md` and tightens or loosens the gates based on observed precision and recall.

C. Comprehension Debt

Code review by a fresh agent is cheap. Comprehension by the operator is expensive. When the loop ships features faster than the operator can understand them, comprehension debt accumulates. At any given moment the operator owes the system more attention than they have to give. The operator who lets comprehension debt grow eventually finds themselves unable to make any architectural decision without the agent's help, which is the moment the loop owns the operator rather than the reverse.

D. Token Blowout

The loop runs forever because no stopping condition was ever defined that can be checked by something other than the agent's own claim. The agent asserts the work is done, the loop terminates, and the half-finished trade sits open. The cure is to insist that every stopping condition is a deterministic inequality over an observable variable: a Sharpe threshold, a drawdown bound, a test suite pass. Never trust the sentence "the agent says it is done."

VIII. The Economics of Compute

A frequently raised objection is that running multiple agents continuously must cost more than the strategies can earn. The objection deserves a concrete answer. Under current API pricing, a loop running a maker on *Sonnet*-class economics and a checker on *Opus*-class economics, averaging five signal candidates per hour and a one-thousand-token verifier trace per candidate, lands in the range of forty to ninety dollars per day in token costs. The cost is real but is not the binding constraint.

The binding constraint is the value of the alpha being generated. A loop managing fifty thousand dollars of capital at an annualized expected return of fifteen percent generates roughly twenty dollars of expected P&L per trading day. At that scale the token bill exceeds the expected P&L and the loop is uneconomic. A loop managing two hundred fifty thousand dollars at the same expected return generates one hundred dollars of expected daily P&L and clears the token cost with margin to spare.

The implication is structural: loop engineering is not yet an economically sensible approach to small accounts. It becomes structurally advantageous somewhere between one hundred thousand and a quarter of a million dollars of deployed capital, and increasingly dominant as scale rises. The institutional advantage erodes not at every level of capital but at the level where token costs become a rounding error against expected P&L.

IX. Conclusion

Loop engineering is not another *XX Engineering* term to be dismissed. It marks the move from the practitioner sitting inside the loop, prompting the agent, to the practitioner standing outside the loop, designing the system that prompts the agent. Applied to quantitative trading—the highest-stakes recursive process available to a private operator—the implications are immediate.

Quantitative trading was always a loop. The institutions that ran it best won because they could afford to put hundreds of researchers and engineers inside it. With a correctly engineered loop, those researchers and engineers become agents, and the headcount collapses to one. The desks that adopt this architecture first will compound for the next decade. The desks still typing prompts one trade at a time will not.

Renaissance compounded a single dollar into twenty-seven thousand by running one loop for thirty years. The opportunity now available to the private operator is to build that loop, on a personal machine, with verification rigor as the only remaining scarce resource. Start with one asset, one signal, one verification rule. Let the loop compound.

Two cautions are worth stating plainly. First, the loop does not manufacture alpha out of nothing. It compounds whatever signal the operator brings into it. Operators who arrive without a research thesis end up with a fast machine for losing money. Second, the verification gates matter more than the maker. A loop with a brilliant maker and a loose checker is a loop that learns to lose efficiently. A loop with a mediocre maker and a strict checker is a loop that compounds slowly and survives.

The practitioners cited throughout this note—Cherny, Steinberger, Osmani, Karpathy—converged on the same observation in the same month from different directions. Each of them was describing the same structural move at a different abstraction layer. Pulled together, the move is this: the entity that ought to be designed is no longer the prompt, no longer the context, no longer the harness. It is the loop that holds all of them. For quantitative trading, that loop is the highest leverage object available to a private operator in 2026.

Acknowledgments

The author thanks the practitioners whose public writing made this synthesis possible: B. Cherny for the original formulation of the loop engineer's mandate; P. Steinberger for the OpenClaw essay that crystallized the term; A. Osmani for the canonical definition; and A. Karpathy for the framing of the operator as bottleneck. Any errors in the synthesis are the author's alone.

X. Toward a Standard Practice

If the framework laid out in this note is correct, the natural next step is for the practitioner community to converge on a small set of standard patterns for loop construction in trading contexts. Three areas are ripe for standardization.

First, the skill file schema. Today every operator invents their own structure for SKILL.md, leading to incompatibility between loops and difficulty in benchmarking. A standard schema with sections for *goal*, *rules*, *lessons*, and *regime tags* would let operators share skill files across architectures and would enable a future generation of evaluators to score skill files for completeness.

Second, the verification suite. The five gates listed in Section III are a starting point, not a finished standard. The quantitative finance community already has a mature literature on backtest evaluation; the work that remains is to compile a canonical verification suite, expressed as deterministic checks any verifier sub-agent can execute, and to publish it under a permissive license.

Third, the incident format. Every operator who has run a loop in production has watched it do something unexpected. The lessons embedded in those incidents are currently locked inside private STATE.md files. A standard incident schema, similar to the post-mortem format that the site-reliability community converged on a decade ago, would let the practitioner community learn collectively rather than in isolation. The first generation of loop operators who publish their incident logs will define the playbook for everyone who follows.

None of these standardizations require any new technology. The primitives all exist. What is missing is a community willing to converge on shared conventions and to publish them as open artifacts. The pattern, once again, mirrors the early evolution of every other engineering discipline: practice precedes formalization, formalization precedes standardization, and standardization precedes the moment when the practice becomes

simply how the work is done.

Appendix: A Minimal Loop Skeleton

The skeleton below is sufficient to begin a paper-trading deployment. Every block can be expanded, but no block can be removed without breaking the recursive self-improvement property the rest of this note argues for.

```
# loop.py
from harness import loop, checker, mcp
from state import state

@loop(interval="1h")
def ingest():
    bars = mcp.market.fetch(
        symbols=UNIVERSE,
        lookback="30d")
    state.write("bars.pq", bars)

@loop(trigger="bars.pq")
def maker():
    sig = agent.run(
        model="sonnet",
        skill="alpha.md",
        data=state.read("bars.pq"))
    state.write("pending.json", sig)

@checker(model="opus")
def gate(sig):
    bt = backtest(sig, window="2y")
    return (
        bt.sharpe > 1.5 and
        bt.max_dd < 0.10 and
        bt.tstat  > 2.0)

@loop(trigger="pending.json")
def execute():
    sig = state.read("pending.json")
    if gate(sig):
        mcp.broker.send(sig, cap=0.02)
        state.append("trades.log", sig)

@loop(interval="1m")
def risk():
    pos = mcp.broker.positions()
    if drawdown(pos) > 0.05:
        mcp.broker.flatten_all()
        state.append("STATE.md",
            "DD breach: positions closed")
```

Fig. 4. The complete loop in roughly thirty lines. Five sub-loops, one persistent state file, one checker, one MCP connector. Everything beyond this is refinement.

References

[1] B. Cherny, “The Job is to Write Loops,” Anthropic Engineering Notes, May 2026.

[2] P. Steinberger, “Stop Prompting Coding Agents,” OpenClaw, Jun. 2026.

[3] A. Osmani, “Loop Engineering: A Definition,” Personal Blog, Jun. 2026.

[4] A. Karpathy, “Remove Yourself as the Bottleneck,” No Priors Podcast, Jun. 2026.

[5] T. Foucault, J. Hombert, and I. Rosu, “News Trading and Speed,” *Journal of Finance*, vol. 71, no. 1, pp. 335–382, 2016.

[6] Anthropic, “Model Context Protocol Specification v1.0,” Technical Report, 2025.

[7] J. Simons et al., “Renaissance Technologies Medallion Fund Performance,” Public Disclosures, 1988–2025.

[8] W. Newey and K. West, “A Simple, Positive Semi-Definite, Heteroskedasticity and Autocorrelation Consistent Covariance Matrix,” *Econometrica*, vol. 55, no. 3, pp. 703–708, 1987.

[9] E. P. Chan, *Algorithmic Trading: Winning Strategies and Their Rationale*. Hoboken, NJ: Wiley, 2013.

[10] C. Greyling, “Loop Engineering: Practical Patterns and Starters,” GitHub repository, MIT License, 2026.